

Where Does Code-Switching Break Knowledge-Graph RAG? Localizing Failures to Entity Linking

Pragya Paramita Pal

Department of Language Science and Technology
Saarland University, Saarbrücken, Germany
prpa00002@stud.uni-saarland.de

Abstract

Most multilingual retrieval papers ask which system gets higher accuracy. But when the database has code-switched text and the system fails it is harder to understand what caused the error, whether it was an entity linking issue or linguistic context effects. A text RAG gets a question, pulls up similar embeddings and does a dense retrieval process to get a document to provide an answer. This distributes language sensitivity throughout the retrieval pipeline. My approach uses a KG-RAG such that for a question, an entity linking process retrieves a Wikidata I.D. (using the Mintaka dataset). After mapping entities in the question to a Wikidata entity I.D., the system no longer cares about language, as the identifier is language-independent. English: diabetes, Hindi: मधुमेह, French: diabète, German: Diabetes would all point to I.D. number Q12206. Mintaka helps us mark exactly where the entity occurs. With this, the system goes on to graph traversal for the ID and outputs facts which are used in the answer generation phase. So KG-RAG gives us an opportunity to check if the language mixture in the database leading to failure or reduced accuracy is a entity-linking problem

1 Introduction

Hindi is chosen for this project because it allows me to look at script changes, romanization and code-switched versions. Japanese allows us these too and it is used as a control setting because its romanization is highly standardized. It lets us check how much Hindi's ambiguous romanization is also affecting the retrieval results. Code switched data can look like:

Romanized: madhumeh ke lakshan kya hain?

Colloquial: sugar ki bimari ke symptoms kya hain?

Mixed: diabetes ke symptoms kya hain?

When retrieval fails, the typical reasons are:

- code-switching distorted the embedding,
- nearest-neighbor search picked the wrong document,

- multilingual alignment collapsed,
- or the answer generator failed.

All of the above are entangled and we cannot tell where the error is caused. With normal RAG benchmarks, it is much harder to distinguish whether the linker had a hard time recognizing the entity or the surrounding sentence. Mintaka identifies which phrase is the identity mention, so a lot of perturbation is possible to understand what breaks. Language mixing perturbations possible are as:

Question	Entity change	Context change
मधुमेह के लक्षण क्या हैं	No	No
madhumeh के लक्षण क्या हैं	Yes	No
मधुमेह ke symptoms kya hain	No	Yes
diabetes ke symptoms kya hain	Yes	Yes

India in Hindi is written as भारत which when romanized, reads "bharat". This is a genuinely native Hindi/Indic name and if linking fails here, we can say that romanization hurt entity linking. The claim is that most KG-RAG failures under code-switching occur because the entity linker cannot find the right Wikidata ID. My aim is to understand where in KG-RAG do code-switching errors occur, and for which types of entity mentions. Also, if alias augmentation can recover entity-linking failures caused by romanized entity mentions. When we perform alias augmentation to go from Q12206-: diabetes/मधुमेह to Q12206-: diabetes/मधुमेह/madhumeh/madhumeha/madhumeh bimari, we check for change in performance. If performance recovers, we would have shown a causal intervention, that localizes failures to the entity-linking stage.

2 Literature Review

Prior work has shown that code-switching poses a challenge for both information retrieval and multilingual language models. Zeng et al. (2026) report substantial degradation when sparse, dense, and late-interaction retrievers are evaluated on code-switched queries, and trace this to divergence between the embeddings of monolingual and code-switched text. Zhang et al. (2023) likewise find that multilingual LLMs do not handle code-switched input reliably. In knowledge-based question answering (KBQA), entity linking has long been recognized as a critical bottleneck: correctly linking mentions to knowledge-graph nodes is typically a prerequisite for answering, and a linking error leaves later stages with no path to the answer (Dubey et al., 2018; Sorokin and Gurevych, 2018). Early work on code-mixed KBQA further points to the importance of surface form, finding that back-transliterating code-mixed questions can outperform translating them (Gupta et al., 2018). More recently, graph-based approaches such as GraphRAG (Edge et al., 2024) and Think-on-Graph (Sun et al., 2024) have demonstrated the value of structured knowledge for retrieval-augmented generation, while multilingual RAG has been explored across languages (Chirkova et al., 2024), though largely without considering code-switching. However, it remains unclear where code-switching causes failures within a multi-stage KG-RAG pipeline. Whereas prior work primarily attributes degradation to retrieval representations, we investigate whether code-switching errors in KG-RAG are concentrated in the entity-linking stage and whether targeted alias augmentation can improve robustness by strengthening that stage alone.

3 Method

3.1 Research Design

A knowledge-graph pipeline has one place where language stops mattering. Entity linking turns a mention into a Wikidata identifier, and from that point on the graph is the same no matter what language the question was asked in. The QID is a pivot.

We use that pivot to locate failures. We fix the target QID and change only the form of the question. Any drop in performance then belongs to a specific stage, not to “retrieval” in general.

A concrete case. The English question “*What man was a famous American author and also a steamboat pilot on the Mississippi River?*” and its Hindi translation both point at Q1497 (Mississippi River). If we ask the Hindi version and the pipeline fails, we want to know whether it failed because it could not find Q1497, or because Q1497’s facts do not contain the answer. Those are different problems with different fixes.

3.2 Data

We use Mintaka, a question-answering benchmark with questions in several languages aligned to Wikidata entities. Two of its properties matter here. It gives gold QIDs, so we always know the right answer. And it marks which span of the question is the entity mention, which is what lets us edit that span and nothing else.

Finding the Hindi mention. Mintaka marks the mention only in English. For “*Who is the president of the United States?*” it records the span *United States* with character offsets into the English string. Those offsets are useless for the Hindi translation, and the English string almost never appears in it.

So we locate the Hindi mention ourselves. We take the entity’s Hindi label and each of its Hindi aliases, and search for them in the translated question, longest first so that a short alias is not matched inside a longer name. If nothing is found, the item has no code-mixed variants and drops out of those comparisons. This succeeds for 854 of 3,725 items (22.9%), which is why the code-mixed conditions have a smaller denominator throughout.

Unusable gold labels. Scoring is done against Wikidata labels, so a broken label guarantees a failure that gets charged to whichever condition the item happens to sit in. We audited all 2,530 gold QIDs and found labels in the wrong script (Q5389, Olympic Games, whose Hindi label is written in Kannada), vandalised English labels (Q2142, reading “Goku is mexican”), and 71 entities with no label in any language. We exclude 159 QIDs on these grounds, leaving 2,371. These items would fail in every condition, so keeping them would add noise, not difficulty.

3.3 Query variants

We want to separate two things that usually travel together: changing the *entity* and changing the *context around it*. So we vary them one at a time. Table 7 lists the result.

Variant	Entity		Frame	
	Lang	Script	Lang	Script
en_latn	en	Latn	en	Latn
hi_deva	hi	Deva	hi	Deva
mix_entity_latn	hi	Latn	hi	Deva
mix_frame_latn	hi	Deva	hi	Latn
hi_latn	hi	Latn	hi	Latn
hi_latn_r	hi	Latn	hi	Latn [†]
cs_mixed	en	Latn	hi	Deva
cs_latn	en	Latn	hi	Latn [†]

Table 1: The eight query variants, defined by the language and script of the entity mention and of the surrounding question frame (en: English, hi: Hindi; Latn: Latin, Deva: Devanagari). The first two come from Mintaka; we build the other six. [†]English borrowings in the frame are restored to their usual English spelling (Section 3.4). Table 8 shows a real item in all eight variants.

Row 3 changes the entity and leaves the context alone. Row 4 does the reverse. Row 5 changes both. Rows 7 and 8 replace the Hindi entity with its English name, which is what a bilingual speaker actually writes. Because every row is built from the same original question, we always compare an item against itself.

Table 8 shows one real item through the pipeline. It is worth reading closely, because it shows three things the design depends on: the entity span survives romanization unrestored (*mississippi nadi* stays as it is), the surrounding frame gets its English borrowings back (*strimbot paylat* → *steamboat pilot*), and the code-mixed row puts the English label in verbatim.

3.4 Building the romanized forms

Mintaka has no romanized Hindi. We generate it. That makes the transliterator part of the experiment, not part of the plumbing: if it makes a systematic mistake, the resulting failures look exactly like the effect we are trying to measure. So we describe it and say how we tested it.

We transliterate Devanagari to Latin using ITRANS, then apply rules for three things ITRANS handles badly:

- **Loanword vowels.** ITRANS has no mapping for ँ, which Hindi uses for English borrowings, so डॉन came out as daँn with the Devanagari character still in it. We map these to their nearest native vowel first.
- **Nasals.** The anusvara ँ assimilates in place.

We write *m* before *p*, *ph*, *b*, *bh*, *m* and *n* elsewhere, so हिंदी gives *hindi* rather than *himdi*, and लंदन gives *landan*.

- **Schwa deletion.** Hindi drops the inherent vowel word-finally and often medially. सबसे is *sabse*, not *sabase*. The medial rule is scoped to exactly three syllables, because an unscoped rule mangles Sanskrit compounds — राष्ट्रपति would become *rashtrpti* instead of *rashtrapati*.

The rules run only on Devanagari spans. An earlier version ran them over the whole string, which rewrote English words sitting in the question: *Iron Man* became *iron nan* and *Real Madrid* became *real nadrid*. We pin all of this with 22 regression tests, and hold the transliterator fixed across every experiment.

Why we also restore loanwords. A bilingual writer code-switches the spelling, not just the word. Nobody types *kailiphorniya ka kaipital kya hai*; they type *California ka capital kya hai*. A literal transliteration therefore produces strings no Hindi speaker would write, which inflates subword fragmentation for a reason that is about our code rather than about romanized Hindi.

So we add a second condition in which English borrowings in the frame are put back into English spelling. Matching is done against the parallel English question, which is how we know what the borrowing was. Plain edit distance is not enough here, because Devanagari romanization and English orthography diverge systematically: क spells *k* where English writes *c*, फ़ spells *ph* where English writes *f*. The literal similarity of *kailiphorniya* to *california* is only 70. We therefore fold both spellings onto a common phonetic key before comparing, which lifts that pair to 100.

The entity span is never passed through this matcher. Its English form is a lookup of the known label, so the entity cannot be corrupted by a fuzzy match. This is also why *hi_latn* and *hi_latn_r* are identical when the only borrowings in an item are inside the entity.

3.5 Grouping entities by how much the names sound alike

Hindi labels relate to English labels in very different ways, and lumping them together hides the effect. Three real cases:

- जेक जिलेनहॉल → *jek jilenahol* for *Jake Gyllenhaal*. The Hindi is a transliteration. Roman-

izing it lands close to the English.

- **मिसिसिप्पी नदी** → *mississippi nadi* for *Mississippi River*. Half transliterated, half translated. The proper noun survives romanization; *nadi* will never match *River*.
- **दूरभाष** → *durbhash* for *telephone*. A Sanskrit coinage. No amount of romanization will connect these two strings, because there is nothing phonetic to connect.

We call this property *phonetic correspondence* and compute it automatically, so it needs no annotator. For each content word of the English label we ask whether any word of the romanized Hindi label is a near match after both have been folded onto the phonetic key described above. Correspondence is the fraction of English content words that find a partner. We count a pair as matching above a similarity threshold τ .

This gives four groups:

Correspond every English word recoverable — *Jake Gyllenhaal*. Romanization should help.

Hybrid partly recoverable — *Mississippi River*. A fixed penalty no transliterator can remove.

Alias-only the label does not correspond but a Wikidata alias does. Q668 is labelled **भारत** (*bharat*), which shares nothing with *India*, but carries the alias **इंडिया**. This is the group alias augmentation exists for.

No-correspond nothing corresponds — *telephone*. Should fail in Devanagari as well as in Latin, which makes it our negative control.

We set $\tau = 72$ and report a sweep over $\tau \in [64, 80]$. This is the only free parameter in the analysis. Across the full sweep the largest group changes by 8% of its own size and the ordering never changes.

3.6 Entity linking

What the linker is given. Candidates are the gold entities of the dataset, about 2,400 of them, not all of Wikidata. Mintaka marks the mention, so the task is to rank candidates against that span. This is easier than open-world linking, and we report it as such: absolute accuracies are upper bounds. The contrasts between variants, which carry our claims, are unaffected.

One consequence has to be stated plainly. In the mention-based conditions the query we hand the linker is the mention, and for *en_latin*, *cs_mixed* and *cs_latin* that mention is the gold English label in all three cases. Those three

conditions are therefore the same query, and a mention-based matcher cannot tell them apart. Code-switching can only be measured in the full-question condition below, and we report it from there alone.

Two matchers. The first is lexical: edit similarity with a phonetic fallback. It compares strings and nothing else. The second is a multilingual bi-encoder (*multilingual-e5-base*), which compares meanings. We run the bi-encoder twice, once on the mention span and once on the whole question. The whole-question condition is the one that resembles ordinary text retrieval, and the only one where the context can influence the query representation rather than just the name.

The two matchers are not redundant. A string matcher cannot see subword fragmentation and cannot translate; a bi-encoder can do both. If romanization hurts the encoder more than the string matcher, the embedding explanation has direct support.

Three indexes.

Base: English labels and English aliases.

Alias: plus Wikidata’s human-written Hindi aliases, in both Devanagari and romanized form.

Oracle: plus the Hindi label itself, in both forms.

Adding both scripts is not a detail. An earlier version added only the romanized forms, which meant every index helped the romanized queries and none could be reached from Devanagari at all. Combined with a phonetic key that stripped non-Latin characters, this put Devanagari at 0% in every group including *CORRESPOND*, where the labels match perfectly — and made romanization appear to improve linking by 73 points.

The *ALIAS* index carries our recovery claim, and it is a fair test because Wikidata editors wrote those aliases, not our transliterator.

The *ORACLE* index is circular on purpose and we report it only as a ceiling. The romanized query for *hi_latin* is the romanized Hindi label, so putting that string in the index guarantees an exact match. To make this visible everywhere, we also report the *trivial-match rate*: how often the query string is literally a key in the index and so needs no matching at all. A reader can subtract it before reading any gain as phonetic recovery.

3.7 Retrieval, and finding where things break

Once an entity is linked we pull its Wikidata facts and write them out as sentences. We fix this to English in every condition. The graph itself is language-independent, but its facts leave as *labels*, and labels have a language — so without this control, language would re-enter the pipeline after the point we are trying to measure.

We then sort every failure into one of three places:

- **Retrieval ceiling.** The subgraph never contained the answer. Nothing upstream could have helped.
- **Entity linking.** The wrong QID was chosen, and the right one would have worked.
- **Verbalisation.** The right QID was chosen but the answer is missing from the written-out facts.

Finally we rerun retrieval with the gold QID substituted in. If performance comes back, the loss was at linking. If it does not, the loss is downstream and the abstract’s claim is wrong. This is the test that decides it.

3.8 Evaluation

Every comparison is within item: the same question in two forms, so the paired difference removes question difficulty. Confidence intervals come from bootstrap resampling over *items*, not rows, because the eight variants of one question are not eight independent observations.

We report top-1 linking accuracy across variants, correspondence groups and indexes. To test the proposed mechanism we add two measurements:

- **Subword fertility** — tokens per word. If romanization hurts by fragmenting words, fertility should predict the penalty.
- **Cross-lingual alignment** — cosine similarity between each variant and the embedding of its own English question. If romanized Hindi falls outside the region the encoder has learned to align, this number should drop.

These two make competing predictions, and the discriminating pattern is whether either tracks the *dense* penalty without also tracking the *lexical* one. A string matcher cannot be affected by tokenization, so a measure that predicts both is standing in for something else, most likely string length.

4 Results

We test whether romanization and code-switching harm KG-RAG, where the damage occurs, and whether aliases repair it.

4.1 There is a hard retrieval ceiling

Substituting the gold QID and retrieving its subgraph yields the gold answer string for 28.6% of items. Label resolution retains 63.3% of harvested claims, and the rest leave the graph as bare Q-numbers that cannot match an answer string.

So 28.6% is the maximum achievable end-to-end score. A condition reaching 28.3% has lost 0.3 points, not 72. We therefore report entity-linking accuracy as the primary measure and use end-to-end sufficiency as a check.

The ceiling is the same in every query variant (Table 4, retrieval-ceiling row, 71.4% throughout), so the English-only verbalisation control worked. All variance between conditions lies inside the 28.6% and all of it is upstream of retrieval.

4.2 Romanization helps lexical matching and harms dense retrieval

This is the central finding. The same manipulation, on the same 3,701 items, moves the two matcher families in opposite directions by comparable amounts.

A lexical matcher compares strings against an English-only index. From Devanagari there is no comparable string, so 0.2% is the expected result and romanization supplies the only usable input.

A dense matcher compares meanings and starts at 36.9% from Devanagari with no augmentation, because *multilingual-e5* has seen a great deal of Devanagari Hindi. Latin-script Hindi is not a conventionalised orthography, so romanizing moves the query to where the model’s cross-lingual alignment is weaker.

The oracle row rules out an index artefact. With both scripts in the index the effect disappears, so the contrast is a property of what the matcher can bridge rather than of what we indexed.

4.3 The effect depends on phonetic correspondence

Romanization only helps when the Hindi and English names already resemble each other. Table 2 stratifies the contrast and gives one entity per group.

Stratum	n	Lexical	Dense
CORRESPOND	1412	0.1 $\rightarrow$ 73.2	54.4 $\rightarrow$ 45.8
HYBRID	391	0.8 $\rightarrow$ 13.8	68.5 $\rightarrow$ 18.2
NO-CORR.	630	0.5 $\rightarrow$ 2.2	49.2 $\rightarrow$ 4.1

Table 2: Accuracy (%) from Devanagari to romanized Hindi by stratum, with the paired lexical difference in brackets. ALIAS-ONLY ($n = 25$) is omitted as too small to interpret. HYBRID transliterates the proper noun and translates the head, so नदी will never match *River*. NO-CORR. entities are coinages and translations, so romanizing दूरभाष to *durbhash* exposes no bridge to *telephone*.

Correspond entities resemble their English labels (e.g., Jake Gyllenhaal), Hybrid entities combine transliteration and translation (e.g., Mississippi River), and No-correspond entities are semantic translations (e.g., for telephone). NO-CORRESPOND is our negative control. Romanization buys 1.8 points, and entity-linking failures number 131 in Devanagari against 129 romanized, a difference of two items in 630. These failures are not script failures: the same items fail in both scripts because the Hindi translation chose a different name. Reporting them as romanization failures would attribute a property of the dataset to the writing system.

The dense column is the mirror image and is the strongest single number we obtained. In HYBRID and NO-CORRESPOND, where no string similarity exists, the dense matcher links Devanagari at 68.5% and 49.2% against 0.8% and 0.5% lexical. It recognises that विश्वयुद्ध denotes *world war* with no characters in common. Romanizing drops those to 18.2% and 4.1%, so romanization removes the capability the multilingual encoder was contributing.

4.4 Context matters more than the entity

A validity note first. In the mention-based arms the query is the mention, and for `en_latin`, `cs_mixed` and `cs_latin` that mention is the gold English label in all three cases. The trivial-match rate is 100% for all three against 3.1% for `hi_latin`. Those arms cannot evaluate code-switching, so we measure it only in the full-question dense arm.

The context carries three times the penalty of the entity. The two factors are additive, summing to -22.3 against -23.6 observed together, and the context cost is unchanged by what the entity is.

What is romanized	Δ	n
Entity only	-5.9	854
Context only	-16.4	854
Both	-23.6	3701
Context, entity romanized	-16.0	854
Context, entity in English	-15.3	854
Loanword spellings restored	$+13.9$	3701

Table 3: Decomposition in the full-question dense arm, paired within item. All intervals exclude zero.

Switching the entity to English does not protect the query from a romanized frame.

This was not the expected outcome. The design assumed the entity mention was where the damage occurred, and only this arm can show otherwise.

Restoring English loanword spellings recovers 13.9 of the 23.6 points. Restoration touches only the frame, so most of the penalty is orthographic convention rather than Latin script as such: *mad-humeh ke symptoms kya hain* retrieves substantially better than *madhumeh ke lakshan kya hain* even though both are romanized Hindi.

4.5 Failures are concentrated at entity linking

Gold-QID injection is the decisive test. Replace the predicted QID with the correct one, rerun retrieval unchanged, and see whether performance returns.

Variant	Predicted	Gold	Recovered
English	28.3	28.6	+0.3
Romanized Hindi	13.1	28.6	+15.5
Hindi (Devanagari)	0.2	28.6	+28.3
<i>Failure stage, % of decisions</i>			
Retrieval ceiling	71.4	71.4	71.4
Entity linking	1.4	28.5	19.9

Table 4: Top panel: retrieval sufficiency (%) under the predicted and gold QID, $n = 3,701$. Bottom panel: stage at which the answer is lost, in the same column order (English, romanized, Devanagari). A verbalisation category is defined but cannot fire, since sufficiency is scored on the predicted QID and a correct link entails sufficiency whenever the gold QID would have sufficed.

Devanagari recovers 28.3 of the 28.4 points available, effectively all of them. English recovers 0.3, because there was nothing to recover. The retrieval-ceiling row is flat at 71.4%, so nothing downstream of the QID is language-sensitive.

Conditional on the retrieval ceiling, every point lost to query form is lost at entity linking. We find

no evidence that graph traversal or verbalisation contributes to the observed degradation.

4.6 Alias augmentation partially repairs the loss

Adding Wikidata’s human-written Hindi aliases in both scripts raises Devanagari linking from 0.2% to 10.1% and romanized linking from 30.0% to 32.3%. The aliases are written by editors rather than by our transliterator, so this is a held-out resource, and the trivial-match rate stays at 5.1%, so the gain is not string identity.

Stratum	Lexical	Dense
ALIAS-ONLY	+13.7	+12.4
HYBRID	+8.2	+8.6
CORRESPOND	+4.3	+2.2
NO-CORRESPOND	+3.5	+0.3

Table 5: Gain (pp) from alias augmentation, pooled over variants. The ordering is the one the correspondence account predicts, in both matchers.

The gain lands where predicted: largest where the label fails but an alias corresponds, smallest where nothing corresponds. Q668 is the clearest case, with the Hindi label भारत sharing nothing with *India* while its alias इंडिया does.

The bound is a property of the knowledge base. Only 12.2% of gold entities carry a Hindi alias, so augmentation cannot reach the other 88%. Even where it suits best, Devanagari reaches 10.1% against the 36.9% a multilingual encoder achieves unaided.

4.7 The dense penalty tracks alignment, not tokenization

Two mechanisms could explain the dense penalty. The discriminating test is whether a measure tracks the dense penalty without also tracking the lexical one, since a string matcher cannot be affected by tokenization.

Romanization moves the query 0.076 further from its own English meaning. Restoring loanword spellings recovers 0.021 of that, matching the accuracy result in Table 3.

Fertility rises as expected but does not explain the penalty. We attribute the dense penalty to degraded cross-lingual alignment and report fertility as inconclusive. Alignment and accuracy come from the same encoder, so a replication with a second encoder family would strengthen this.

Measure	Value
<i>Cosine to own English question</i>	
Devanagari	0.865
Romanized	0.789
Romanized, restored	0.811
Within-item drop	−0.076
<i>Fertility (tokens per word)</i>	
Devanagari → romanized	1.61 → 1.77
<i>r</i> with dense penalty	0.18
<i>r</i> with lexical penalty	0.12
<i>r</i> , dense full question	0.01

Table 6: Alignment and tokenization evidence. Alignment is supported: romanization lowers similarity to the English question, restoring loanwords recovers part of it, and the cosine ordering across all seven variants matches the full-question dense accuracies on six of seven. Fertility is not separated: it correlates almost as strongly with the lexical penalty as with the dense one, which suggests it tracks string length, and the arm with the largest penalty shows the weakest correlation.

5 Takeaway

Romanization improves lexical entity linking by 29.8 points and harms dense multilingual linking by 16.6 points on the same items, so the direction of the effect is a property of the linker and not of the language. Its size is governed by phonetic correspondence: 73.1 points where the names correspond and 1.8 where the Hindi name is a translation, in which case linking fails in Devanagari too. Almost all resulting degradation is localised to entity linking, where human-written aliases recover part of the loss but reach only an eighth of the entity inventory.

Ethics Statement

I’ve used generative AI tools to refine research hypotheses, obtain feedback on methodology and experimental design, suggest experimental parameters, assist with software implementation, and support the interpretation of results.

References

- Nadezhda Chirkova, David Rau, Hervé Déjean, Thibault Formal, Stéphane Clinchant, and Vassilina Nikoulina. 2024. [Retrieval-augmented generation in multilingual settings](#). In *Proceedings of the 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024)*, Bangkok, Thailand. Association for Computational Linguistics.
- Mohnish Dubey, Debayan Banerjee, Debanjan Chaudhuri, and Jens Lehmann. 2018. [EARL: Joint entity and relation linking for question answering over](#)

knowledge graphs. In *The Semantic Web – ISWC 2018*, volume 11136 of *Lecture Notes in Computer Science*, pages 108–126. Springer.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. [From local to global: A graph RAG approach to query-focused summarization](#). *arXiv preprint arXiv:2404.16130*.

Vishal Gupta, Manoj Chinnakotla, and Manish Shrivastava. 2018. [Transliteration better than translation? answering code-mixed questions over a knowledge base](#). In *Proceedings of the Third Workshop on Computational Approaches to Linguistic Code-Switching*, pages 39–50, Melbourne, Australia. Association for Computational Linguistics.

Daniil Sorokin and Iryna Gurevych. 2018. [Mixing context granularities for improved entity linking on question answering data across entity categories](#). In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics*, New Orleans, Louisiana. Association for Computational Linguistics.

Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. [Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph](#). In *The Twelfth International Conference on Learning Representations*.

Qingcheng Zeng, Yuheng Lu, Zeqi Zhou, Heli Qi, Puxuan Yu, Fuheng Zhao, Hitomi Yanaka, Weihao Xuan, and Naoto Yokoya. 2026. [Code-switching information retrieval: Benchmarks, analysis, and the limits of current retrievers](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 13055–13071, San Diego, California, United States. Association for Computational Linguistics.

Ruochen Zhang, Samuel Cahyawijaya, Jan Christian Blaise Cruz, Genta Indra Winata, and Alham Fikri Aji. 2023. [Multilingual large language models are not \(yet\) code-switchers](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore. Association for Computational Linguistics.

A Appendix

	Variant	Example Entity	Context
1	en_latin	What English are the symptoms of diabetes?	English
2	hi_deva	मधुमेह के लक्षण क्या हैं	Devanagari
3	mix_entity_latin	madhumeह के लक्षण क्या हैं	Romanized Devanagari
4	mix_frame_latin	मधुमेह ke lakshan kya hain	Hindi Romanized
5	hi_latin	madhumeह ke lakshan kya hain	Romanized Devanagari
6	hi_latin_r	madhumeह ke symptoms kya hain	Romanized [†] Devanagari
7	cs_mixed	diabetes के लक्षण क्या हैं	English Devanagari
8	cs_latin	diabetes ke symptoms kya hain	English Romanized [†]

Table 7: The eight query variants, shown on a schematic example for legibility. Rows 1 and 2 come from Mintaka; rows 3–8 we build. [†]English borrowings restored to their usual English spelling (Section 3.4). Table 8 shows a real item.

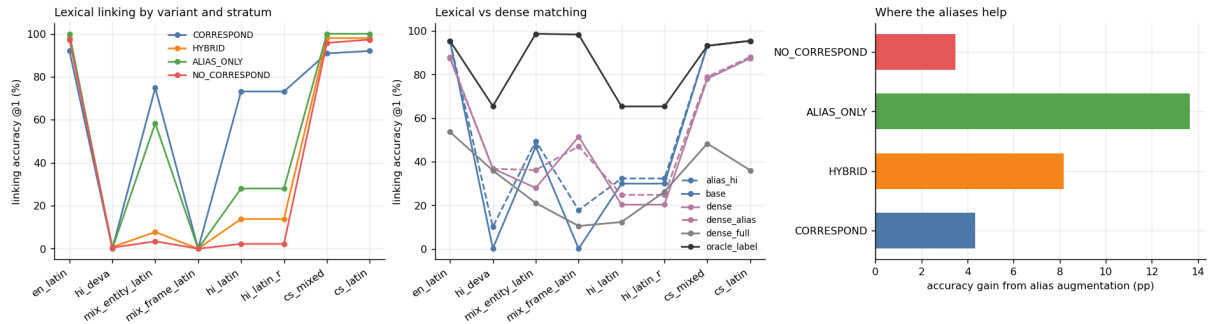


Figure 1: Entity linking accuracy at rank 1 across the eight query variants. **Left:** the lexical matcher by correspondence stratum. Devanagari entities (hi_deva, mix_frame_latn) are never linked, since an English-only index offers no comparable string. Romanizing the entity helps only where the Hindi and English names sound alike: CORRESPOND rises to about 75%, while NO-CORRESPOND stays near 2%. **Middle:** lexical (blue) and dense (purple, grey) matchers, with and without Hindi aliases (dashed). Romanization moves them in opposite directions: the lexical matcher gains and the dense matcher loses. The black line is the oracle index, which contains the Hindi label itself and serves only as a ceiling. **Right:** gain from adding Wikidata’s Hindi aliases to the lexical index. The gain is largest for ALIAS-ONLY entities, whose label fails but whose alias corresponds. In the mention-based arms, the high scores for cs_mixed and cs_latn reflect a trivial match, since the mention is the English label itself. The mix_ variants cover only the 854 items where the Hindi mention could be located.

en_latn	What man was a famous American author and also a steamboat pilot on the Mississippi River ?
hi_deva	प्रमुख अमेरिकन लेखक और साथ ही मिसिसिप्पी नदी में एक स्टीमबोट पायलट कौन हैं?
hi_latn	pramukh amerikan lekhak aur sath hi mississippi nadi men eka strimbot paylat kaun hain?
hi_latn_r	pramukh american lekhak aur sath hi mississippi nadi men eka steamboat pilot kaun hain?
cs_latn	pramukh american lekhak aur sath hi Misissippi River men eka steamboat pilot kaun hain?

Table 8: One Mintaka item (fae46b21, gold entity Q1497, *Mississippi River*) in five of the query variants, as our pipeline produces them. Bold marks the entity mention and the English borrowings in the frame. en_latn and hi_deva are Mintaka’s English question and its Hindi translation. hi_latn is our literal romanization of the Hindi, so borrowings come out in Hindi spelling (*strimbot paylat*). hi_latn_r restores those borrowings to English spelling (*steamboat pilot*, *american*) but leaves the entity untouched, so that only the frame changes between the two rows. cs_latn keeps the romanized frame and replaces the entity with its English label, as a bilingual writer would. The entity is a HYBRID case: the proper noun is transliterated (*mississippi*) but the head is translated (*nadi*, “river”), so romanization alone cannot recover the English label.